

TCS Technical Interview — Project Q&A

*Storm-Aware Ionospheric TEC / Delay Prediction using LSTM & Transformer (Bangalore GNSS Data)
— detailed answers for a 2027-passout technical round*

1. Project Overview

Q1. Tell me about your project.

I worked on a storm-aware ionospheric delay prediction system for GNSS (GPS) signals.

Situation

Global Navigation Satellite System (GNSS) is an umbrella term for satellite constellations—such as GPS, GLONASS, Galileo, and BeiDou—that provide global spatial positioning. Regional - NavIC (INDIA)

GNSS (GPS) signals travel from satellites through the ionosphere — a layer of the atmosphere containing free electrons — before reaching a ground receiver. As they pass through, these charged particles slow the signal down, causing what's called ionospheric delay, which directly translates into positioning error on the ground. On a normal "quiet day," this delay is fairly stable and predictable. But the Sun periodically releases Coronal Mass Ejections (CMEs) — huge bursts of solar plasma and magnetic field — that hit Earth's magnetosphere and trigger geomagnetic storms. On a "storm day," the ionosphere's electron content becomes highly disturbed and non-linear, causing GPS/GNSS delay to spike unpredictably, which can degrade positioning accuracy for safety-critical systems like aviation, precision agriculture, and autonomous navigation. Existing prediction models in the literature were mostly trained on quiet-time or full-solar-cycle data, so they weren't reliable specifically during severe storms — which is exactly when accurate forecasts matter most.

Task

My target was to build a storm-aware forecasting system that could predict next-day ionospheric delay accurately enough to be useful during real geomagnetic storms — not just on calm days. Specifically, I set out to:

- (1) quantify how differently the ionosphere behaves on a quiet day versus a storm day using real geomagnetic indices,
- (2) build models that forecast GNSS delay for both GPS frequencies (L1 and L5)
- (3) determine which deep-learning architecture holds up best as storm severity increases, so it could realistically feed into an early-warning system for accidental positioning error.

Action

I collected a full year (2024) of minute-resolution TEC data from the IISc Bangalore ground station — a location near the Equatorial Ionization Anomaly crest, which makes it especially sensitive to storm effects — and converted TEC into ionospheric delay using the standard frequency-scaling formula for L1 and L5. To properly separate quiet-day and storm-day behavior, I classified days using three independent indices — Dst, Kp, and Ap — and specifically picked out three real CME-driven storms from May 2024 of increasing severity: a G2 (moderate), G3, and G5 (the strongest storm in roughly the

last decade), each paired with its preceding quiet day for comparison. I trained the model on 334 quiet days and deliberately held out these storm days as validation, rather than doing a random split, so the evaluation would reflect real storm-time performance. I built and compared two deep learning models — an LSTM and a Transformer with a global attention mechanism — and evaluated them with both regression metrics (RMSE, correlation) and event-detection metrics (POD, FAR, CSI, F1) at multiple delay thresholds, since average accuracy alone can hide poor performance on the large, rare spikes that matter operationally.

Result

The comparison clearly showed that quiet-day and storm-day ionospheric behavior are fundamentally different — correlation and RMSE degraded for both models as storm severity increased from G2 to G5, confirming that CME-driven disturbances genuinely break the assumptions of quiet-time models. But the Transformer consistently outperformed the LSTM, especially on the strongest G5 storm, where its long-range attention mechanism handled the complex, non-linear delay dynamics noticeably better — giving a stronger correlation and better event-detection scores. This gave a concrete, data-backed recommendation: a Transformer-based model is the more reliable choice as the core of a storm-aware GNSS early-warning system, which could help flag reduced positioning accuracy in advance for GPS-dependent applications.

Answer: I worked on a storm-aware ionospheric delay prediction system for GNSS (GPS) signals. GNSS signals slow down as they pass through the ionosphere — this is called ionospheric delay — and the amount of delay depends on the Total Electron Content (TEC) along the signal path. I collected a full year (2024) of minute-resolution TEC data from the IISc Bangalore ground station (part of the International GNSS Service network), converted TEC into ionospheric delay for the L1 and L5 GPS frequencies using the standard delay equation, and built two deep learning models — an LSTM and a Transformer — to forecast next-day delay. I specifically validated both models on three real geomagnetic storms of different severity that occurred in May 2024 (G2, G3, G5, as classified by NOAA), because ionospheric behavior becomes highly unpredictable during solar storms, which is exactly when GNSS positioning errors matter most. I evaluated the models using regression metrics (RMSE, correlation coefficient) as well as event-detection metrics (POD, FAR, CSI, F1-score) at multiple delay thresholds, and found the Transformer performed better than the LSTM, especially during the most severe storm.

Q2. What was the main objective of your project?

Answer: My target was to build a storm-aware forecasting system that could predict next-day ionospheric delay accurately enough to be useful during real geomagnetic storms — not just on calm days. Specifically, I set out to:

- (4) quantify how differently the ionosphere behaves on a quiet day versus a storm day using real geomagnetic indices,
- (5) build models that forecast GNSS delay for both GPS frequencies (L1 and L5)
- (6) determine which deep-learning architecture holds up best as storm severity increases, so it could realistically feed into an early-warning system for accidental positioning error.

Q3. Why did you choose this project?

Answer: GNSS/GPS accuracy directly affects real-world systems like aviation, precision agriculture, surveying, and navigation, and the ionosphere is one of the largest sources of error in satellite positioning — especially during space-weather events. I wanted a project that combined applied deep learning (time-series forecasting with

LSTM/Transformer) with a real physical system and measurable, interpretable outputs, rather than a purely synthetic dataset. It also let me work with a genuinely difficult forecasting problem, since storm-time ionospheric behavior is highly non-linear.

Q4. What problem does your project solve?

Answer: It solves the problem of not knowing, in advance, how much error a GPS-based system will experience due to the ionosphere — particularly during a geomagnetic storm, when the delay can spike unpredictably. By forecasting delay ahead of time, downstream systems can flag a potential accuracy drop or apply a correction before it causes a real positioning error.

Q5. Who are the intended users of your project?

Answer: The end beneficiaries would be systems and organizations that depend on precise GNSS positioning — aviation navigation systems, precision-agriculture platforms, surveying and mapping services, autonomous vehicle positioning stacks, and space-weather/early-warning agencies (like ISRO's Space Application Center, which co-funded the related research work) that need to alert downstream GNSS users during storm conditions.

Q6. Tell me about the future scope of your project.

Answer: Future scope includes: (1) adding exogenous space-weather driver features such as Kp, Dst, Ap and solar wind speed as direct model inputs so the model gets early signal of an incoming storm rather than inferring it purely from past TEC; (2) extending from a single station (Bangalore) to a multi-station network with a shared model conditioned on station location; (3) training across multiple years to cover different solar-cycle phases, since 2024 data alone is near solar maximum; (4) building a real-time ingestion and alerting pipeline so predictions can actually feed a live early-warning system instead of only being validated offline.

2. Your Role

Q7. What was your role in the project?

Answer: I was responsible for the end-to-end machine learning pipeline: collecting and cleaning the raw TEC data, converting it into ionospheric delay values, designing and implementing both the LSTM and Transformer models in PyTorch, running the storm-specific training/validation experiments, computing all evaluation metrics, and refactoring the whole pipeline from a single working notebook into a modular, reusable codebase.

Q8. Was it an individual or team project?

Answer: State this honestly based on your situation — if it was done as part of a research group (the associated conference paper lists multiple co-authors from Narula Institute of Technology and JIS University), describe it as a team research project where you led/owned the machine-learning modeling and pipeline-engineering side, while being ready to name what other members contributed (e.g., data collection, literature review, paper writing) if asked. If you built the code independently, say so directly — TCS interviewers value honesty here more than the answer itself.

Q9. What specific modules did you develop?

Answer: I developed: (1) the data-sorting module that converts raw per-satellite CSV files into minute-wise TEC series, (2) the TEC-to-ionospheric-delay conversion module (frequency scaling plus an elevation-based mapping function), (3) the LSTM model module (a 2-layer stacked LSTM with a linear output head), (4) the Transformer model module (with a global attention encoder and a warmup-cosine learning-rate schedule), (5) the training loops for both models, and (6) the evaluation/plotting module that generates the actual-vs-predicted plots and computes RMSE, correlation, POD, FAR, CSI and F1.

Q10. How did you coordinate with your teammates?

Answer: If done with a team: through regular syncs to align on the shared dataset format and evaluation methodology, dividing work by concern (e.g., someone handling literature review and paper writing while you owned the modeling code), and using version control / shared folders to avoid conflicting changes to the pipeline. Be ready to name the actual tool you used (e.g., Git, shared Drive, WhatsApp/Slack for sync) truthfully.

Q11. What challenges did you personally face?

Answer: The biggest personal challenge was that my initial LSTM model produced an overly smoothed prediction curve that missed the sharp delay spikes during storms — an important failure since spikes are exactly what an early-warning system needs to catch. I diagnosed this as a consequence of using a plain MSE loss with only single-day input context, which under-weights rare large deviations, and addressed it by building and comparing a Transformer with a global attention mechanism, which handled long-range storm dynamics noticeably better on the most severe (G5) storm day.

Q12. Suppose you are group leader, among your teammates some are not working properly, how would you handle the situation?

Answer: I would first have a private, non-confrontational one-on-one conversation to understand why — it could be unclear task scope, being stuck on a technical blocker, or a personal issue — rather than assuming a lack of effort. If it's a scope/clarity problem, I'd break their task into smaller, well-defined pieces with clear deadlines. If it's a skill gap, I'd pair them with a stronger teammate or offer to walk through the blocker together.

Answer: If the issue persists despite support, I would set clear, agreed-upon milestones and check in regularly, and escalate to a mentor/faculty guide only as a last resort, after giving a fair chance to improve. The goal is to protect both the project timeline and the team relationship, not to publicly blame anyone.

3. Technical Questions

Q13. Which programming language(s) did you use, and why?

Answer: Python, because it has the strongest ecosystem for both data processing (pandas, numpy) and deep learning (PyTorch), and it made it easy to move from data cleaning to model training to evaluation within one consistent codebase.

Q14. What technologies, frameworks, or tools did you use?

- PyTorch — for building and training both the LSTM and Transformer models
- pandas / numpy — for TEC data loading, sorting, and numerical processing
- matplotlib — for actual-vs-predicted plots and loss curves
- Jupyter Notebook — for experimentation, later refactored into a modular src/ package (separate config, data-loading, preprocessing, model, training, plotting, and utils modules) with the notebook reduced to a thin orchestration layer
- NVIDIA RTX 3060 GPU (12GB) with 64GB RAM on an AMD Ryzen 5 7600X — for local model training

Q15. Why did you choose those technologies over alternatives?

Answer: PyTorch over TensorFlow/Keras because of its more Pythonic, imperative style which makes debugging custom architectures (like the Transformer's global attention encoder) easier, and because it's the current standard in most recent time-series and NLP research I referenced. pandas/numpy were the natural choice for structured minute-resolution time-series data over, say, a full database engine, since the dataset size (a year of per-minute data for one station) comfortably fits in memory and doesn't need query-based access patterns.

Q16. Which database did you use? Why?

Answer: This project did not use a traditional relational or NoSQL database — the data was stored and processed as sorted CSV files and pandas DataFrames, since the dataset (about 365 days of minute-resolution TEC values for one station) is small enough to fit in memory and doesn't need transactional guarantees or concurrent multi-user access. If this were extended into a production system ingesting live data from many GNSS stations continuously, I would introduce a time-series database (like InfluxDB or TimescaleDB) suited to high-frequency sensor-style data, or a data lake (e.g., Parquet files on cloud storage) for scalable batch processing.

Q17. Explain your project architecture.

Answer: At a high level it's a three-stage pipeline: (1) Data layer — raw per-satellite TEC CSV files are discovered, sorted chronologically, and reduced to a single minute-wise TEC series per day; (2) Processing layer — TEC is converted to ionospheric delay for L1 and L5 using the frequency-scaling formula and an elevation-based mapping function, and sequences are constructed for model input; (3) Modeling layer — the same processed sequences are fed

independently into an LSTM model (two stacked LSTM layers plus a linear output head) and a Transformer model (with a global attention encoder), each trained with its own training loop, and both are evaluated with a shared metrics/plotting module.

Answer: The codebase itself is modularized into a `src/` directory (configs, loaders, preprocessing, models, training, plots, utils) with a slim orchestration notebook that calls into these modules — this was a deliberate refactor from an earlier single-notebook prototype to make the pipeline reusable across two separate datasets.

Q18. How does the data flow through your application?

Answer: 1) Raw per-satellite TEC CSVs (named by day, e.g., `iisc<day>_TECU`) are read and sorted into a minute-wise maximum-TEC series per day.

Answer: 2) That sorted series is converted into ionospheric delay in meters, separately for the L1 and L5 frequencies, using the ionospheric delay formula.

Answer: 3) The delay sequences are windowed into (input sequence → next-value) pairs and wrapped in PyTorch TensorDatasets/DataLoaders.

Answer: 4) The LSTM and Transformer models each consume these batches during training, producing a trained model checkpoint plus a loss history.

Answer: 5) On specific storm validation days, the trained models forecast delay, which is compared against the actual observed delay to produce plots and the full metrics table (RMSE, correlation, POD, FAR, CSI, F1).

Q19. How did you handle exceptions and errors?

Answer: During data ingestion I added explicit checks for malformed folder names and missing files (using regex-based folder discovery so any folder that doesn't match the expected `'iisc<day>_TECU'` pattern is safely skipped rather than crashing the pipeline). I also fixed a real bug during development where a CSV overwrite issue was silently losing previously processed data — I caught this by comparing output row counts before and after a pipeline run and added guards to prevent unintended overwrites. For the training loop, I used try/except around file I/O and validated tensor shapes before feeding batches into the model to fail fast with a clear message rather than a cryptic PyTorch shape-mismatch error deep in the stack.

Q20. Prepare interview questions from the technologies you used in the project (guidance).

Answer: Since you used PyTorch, pandas, numpy, and matplotlib, be ready for direct technology questions such as: 'What is the difference between a DataLoader and a Dataset in PyTorch?', 'How does autograd work?', 'Why use `batch_first=True` in an LSTM?', 'What is the difference between `.loc` and `.iloc` in pandas?', 'How do you avoid a `SettingWithCopyWarning`?', and 'How would you vectorize a loop-based pandas operation with numpy?'. Practice answering these independently of the project, since technical rounds often branch into general tool questions once you mention a library.

4. Design and Implementation

Q21. Explain the workflow of your project.

Answer: Collect raw TEC data for the full year → sort and clean it into per-day minute-resolution series → convert TEC to ionospheric delay for L1 and L5 → hold out May 2024 as the storm validation period while training on the remaining 334 days → train the LSTM and Transformer models independently on the training data → forecast delay for the three specific storm days (G2, G3, G5) → compare predicted vs actual delay visually and with regression + event-detection metrics → conclude which model generalizes better to storm conditions.

Q22. Describe the database schema.

Answer: There isn't a relational database schema in this project since the data is file-based (CSV/DataFrame), not stored in a DB engine. The logical 'schema' of the processed data is a per-day time series with columns like timestamp (minute resolution), TEC value, and derived ionospheric delay in meters for L1 and L5. If asked to design a schema for a production version, I'd propose a time-series table keyed by (station_id, timestamp) with columns for TEC, delay_L1, delay_L5, and a separate table for geomagnetic index values (Kp, Dst, Ap) keyed by timestamp, joined on time for storm-aware feature engineering.

Q23. What algorithms or data structures did you use?

Answer: LSTM (Long Short-Term Memory) — a recurrent neural network architecture using gated memory cells (forget/input/output gates) to model sequential dependencies; implemented as two stacked nn.LSTM layers followed by a linear output layer.

Answer: Transformer encoder with self-attention — including a global attention encoder to capture long-range dependencies across the input window, trained with a warmup-cosine-decay learning-rate schedule.

Answer: Standard data structures: pandas DataFrames/Series for time-indexed data, PyTorch Tensors and TensorDatasets for batched model input, and Python dictionaries for configuration (e.g., mapping dataset/day numbers to calendar dates).

Q24. How did you optimise your project?

Answer: On the modeling side: increasing Transformer depth, adding a global attention encoder, and using a warmup-cosine-decay learning-rate schedule instead of a fixed learning rate improved training stability and final accuracy; I also increased LSTM training epochs to 50 after observing it was still improving. On the engineering side: refactoring the single monolithic notebook into a modular src/ package removed duplicated code across two datasets and made re-running experiments faster and less error-prone.

Q25. How did you ensure security?

Answer: This is a research/data-science pipeline rather than a user-facing application, so there's no authentication or user-data-security surface in the current version. What I did ensure was data integrity — validating that sorted output files matched expected row counts, guarding against accidental overwrite of processed CSVs, and keeping raw and processed data in clearly separated folders so a bug in one stage couldn't silently corrupt another. If this became a live service (e.g., an API serving delay forecasts), I would add authentication/rate-limiting on the API layer and input validation on any externally-supplied coordinates or timestamps.

Q26. Did you use APIs? Which ones and why?

Answer: The core modeling pipeline itself doesn't call external APIs — it reads pre-downloaded TEC data from the IGS network and geomagnetic index data referenced from spaceweatherlive.com and the Kyoto World Data Center for Dst. If deployed as a real early-warning product, I would expose the model through a REST API (e.g., a FastAPI service) so downstream navigation/GNSS systems could query predicted delay for a given station and time, and would consume live geomagnetic index APIs as real-time input features.

5. Testing

Q27. How did you test your project?

Answer: I validated the pipeline in stages: verifying the data-sorting step produced the correct number of rows per day, visually inspecting actual-vs-predicted plots for each model on each storm day, and cross-checking storm-day classification against three independent geomagnetic indices (Dst, Kp, Ap) rather than trusting a single indicator, to make sure the 'G2/G3/G5' labels were correct before drawing any conclusions from the model comparison.

Q28. What types of testing did you perform?

Answer: Unit-level sanity checks — e.g., confirming the TEC-to-delay conversion formula produced physically reasonable delay values (a few to a few tens of meters) before feeding them into training.

Answer: Pipeline/integration testing — running the full sorting → conversion → training → evaluation flow end-to-end on a small subset of days first, before running it on the full year.

Answer: Model validation — held-out, storm-specific validation days (not a random split) to test real forecasting performance, plus loss-curve inspection (training vs validation loss) to check for overfitting.

Answer: Quantitative evaluation — RMSE, correlation coefficient, POD, FAR, CSI, and F1-score at multiple delay thresholds (3m, 8m, 15m) to test both average accuracy and rare-event detection ability.

Q29. Did you encounter any bugs? How did you fix them?

Answer: Yes — three concrete ones: (1) a folder-naming bug where inconsistent naming broke the regex-based folder discovery, fixed by tightening the pattern and adding explicit validation; (2) an incorrect label-derivation bug where the day/date mapping was misaligned for one dataset, fixed by cross-checking against the known calendar dates of the storms; (3) a CSV overwrite bug where re-running the pipeline silently overwrote previously processed data instead of appending or versioning it, fixed by adding explicit output-path checks and row-count validation before/after each run.

Q30. How did you verify the accuracy of the results?

Answer: By using multiple independent metrics that each test a different aspect of accuracy: RMSE and correlation coefficient for how close and well-timed the overall predicted curve is, and POD/FAR/CSI/F1 (at 3m, 8m, and 15m thresholds) for whether the model correctly detects the specific large delay events that matter operationally. Agreement across both metric families (e.g., the Transformer winning on both correlation and CSI for the G5 storm) gave more confidence than any single number would.

6. Challenges

Q31. What was the biggest challenge during development?

Answer: Getting a model to actually forecast the sharp storm-time delay spikes instead of a smoothed, lagging curve. My initial LSTM setup, trained with a plain MSE loss on single-day input context, systematically under-predicted rare large deviations because they're a small fraction of the total training signal. Recognizing this as a spike-detection problem (not just a generic regression problem) and testing a Transformer with global attention as an alternative was the key turning point.

Q32. How did you resolve technical issues?

Answer: By isolating the problem before jumping to a fix — first visually confirming the smoothing behavior in the actual-vs-predicted plots, then reasoning about the cause (loss function + limited context window), then testing one change at a time (architecture, learning-rate schedule, training length) so I could attribute improvement to the right cause instead of changing everything simultaneously.

Q33. What would you improve if you had more time?

Answer: Add exogenous space-weather features (Kp, Dst, Ap, solar wind speed, F10.7 flux) as direct model inputs instead of relying only on past TEC.

Answer: Use a spike-aware loss function (e.g., Huber loss or a custom weighted loss) so large deviations are penalized more heavily during training.

Answer: Extend training to multiple years to cover different solar-cycle phases, not just the near-solar-maximum year 2024.

Answer: Extend from a single station to a multi-station model so results generalize beyond Bangalore.

Q34. What did you learn from this project?

Answer: Beyond the technical skills (PyTorch, sequence modeling, attention mechanisms), I learned the importance of designing evaluation around the actual failure mode you care about — a model can look accurate on average (low RMSE) while still failing at the rare, high-impact events that matter most, which is why the event-detection metrics (POD/FAR/CSI) turned out to be just as important as the regression metrics. I also learned the practical value of refactoring research code into a modular, reusable pipeline early rather than late.

7. Practical Questions

Q35. Can you explain your code for a particular module? (e.g., the LSTM model)

Answer: The LSTM model class defines two stacked `nn.LSTM` layers — the first takes a single input feature (the delay value at each time step) and produces a hidden representation of size `units_1`, the second takes that hidden sequence and produces a representation of size `units_2`, and a final `nn.Linear` layer maps that down to a single predicted delay value per time step. In the forward pass, the input tensor flows through `lstm1`, its output sequence flows through `lstm2`,

and the final fully-connected layer produces the prediction. This stacked structure lets the first layer learn shorter-term local patterns and the second layer combine them into higher-level temporal structure.

Q36. If I remove one component, what will happen?

Answer: For example, if the elevation-based mapping function is removed from the TEC-to-delay conversion, the delay values would only reflect the vertical (zenith) component of TEC rather than the true slant path a satellite signal actually travels, making the delay values systematically underestimated for low-elevation satellites — this is analogous to how removing a preprocessing step introduces a hidden bias into a pipeline rather than causing an obvious crash, which is why validating intermediate outputs (not just final metrics) matters.

Q37. How scalable is your application?

Answer: As a research pipeline processing one station's yearly data on a single GPU, it's not designed for high concurrency, but the architecture would scale reasonably well: the data-processing stage is already modular per dataset/station, so adding more stations mainly means running the same pipeline with different input folders, and the model layer could be extended to a shared model conditioned on station identity rather than training one model per station, which keeps compute cost roughly linear rather than exploding with the number of stations.

Q38. How can your project handle more users? (framed as: more stations / more concurrent inference requests)

Answer: For more simultaneous prediction requests, I would move from ad-hoc notebook execution to a served model behind a lightweight inference API (e.g., FastAPI + a model server), add batching so multiple requests share a forward pass where possible, and cache recent predictions per station since ionospheric delay changes gradually minute-to-minute, so identical near-term requests don't need to be recomputed from scratch every time.

Q39. What are the limitations of your project?

Answer: Trained and validated on a single station (Bangalore) and a single year (2024, near solar maximum) — results may not generalize to other locations or solar-cycle phases without retraining.

Answer: Validated on only one instance of each storm class (one G2, one G3, one G5 day), which limits statistical confidence in the storm-time performance numbers.

Answer: No exogenous space-weather features are used as model input — the model infers storm effects purely from past TEC/delay values.

Answer: Not yet deployed as a live, real-time system — evaluation is entirely offline/retrospective.

8. Future Enhancements

Q40. What features would you add in the future?

Answer: Exogenous space-weather driver inputs (Kp, Dst, Ap, solar wind speed), multi-day input context instead of single-day windows, and a hybrid ensemble that combines LSTM and Transformer predictions, weighting the Transformer more heavily during detected high-Kp periods.

Q41. How can your project be made more efficient?

Answer: Model-side: mixed-precision training to reduce GPU memory and speed up training; a lighter attention variant (e.g., a local/windowed attention) to reduce the Transformer's quadratic compute cost if input windows grow larger. Pipeline-side: caching intermediate sorted/processed data so repeated experiments don't re-run the full data-sorting step each time.

Q42. How would you deploy your project in a real-world environment?

Answer: Wrap the trained model in an inference API (FastAPI/Flask) that accepts a station ID and returns the next-period predicted delay.

Answer: Containerize it with Docker for consistent deployment and set up a scheduled batch job (e.g., every 5–15 minutes) that pulls the latest TEC and geomagnetic index data and refreshes predictions.

Answer: Add a monitoring layer tracking live prediction error against the offline-validated baseline, with alerts if error drifts significantly (a sign of concept drift, e.g., moving into a different solar-cycle phase).

Answer: Expose predictions through a dashboard or downstream alert system that flags elevated-risk periods to GNSS-dependent applications.

Scenario-Based Questions

Q43. Suppose your database crashes. How will your application recover?

Answer: Since the current pipeline is file-based rather than a live database-backed service, the direct risk is corrupted or lost processed CSVs — mitigated by keeping raw source data untouched and separate from processed/derived data, so any processed output can be regenerated by simply re-running the pipeline. In a production version with a real time-series database, I'd rely on regular automated backups/snapshots and a replica/failover instance, and design the ingestion job to be idempotent (safe to re-run) so a crash mid-write doesn't corrupt state.

Q44. What if your API server becomes unavailable?

Answer: For a production inference API, I'd deploy at least two replicas behind a load balancer so a single instance failing doesn't take down the service, add health checks so an unhealthy instance is automatically removed from rotation, and have the client/downstream system fall back to the last known-good prediction (with a clear staleness flag) rather than failing hard, since a slightly stale ionospheric delay forecast is still more useful than no forecast at all.

Q45. How would you improve the performance of your application?

Answer: On the model side: the spike-aware loss and exogenous feature improvements described earlier. On the systems side: batching inference requests, caching near-term predictions, and moving from CPU-bound data preprocessing to vectorized numpy/pandas operations (already largely done) or GPU-accelerated preprocessing if the data volume grows substantially (e.g., many GNSS stations at once).

Q46. If 10,000 users access your application simultaneously, what changes would you make?

Answer: Move from a single-instance inference script to a horizontally-scaled deployment (multiple API replicas behind a load balancer), add a caching layer (e.g., Redis) for predictions that don't change second-to-second, use asynchronous request handling so I/O-bound requests don't block the event loop, and add autoscaling so instance count adjusts to real-time load rather than being fixed.

Q47. How would you secure user data?

Answer: Enforce HTTPS/TLS for all API traffic, authenticate requests with API keys or OAuth tokens depending on the consumer type, apply rate-limiting per client to prevent abuse, and — since this project's own data is public GNSS/space-weather data rather than personal user data — the main data-protection concern in a real deployment would shift to protecting the service itself (availability, integrity of predictions) rather than personal data confidentiality, but any user account or subscription data added on top (e.g., alert-subscription emails) would be encrypted at rest and access-controlled.

Q48. How to tune performance of your project in future?

Answer: Systematically: run learning-rate and batch-size sweeps for both models, experiment with input-window length (currently single-day) to find the point where more historical context stops improving accuracy but does increase compute cost, and use the existing storm-specific validation setup (not a random split) as the ground truth for every tuning decision, since that's the scenario the project actually cares about.

HR + Project Combined Questions

Q49. Why are you proud of this project?

Answer: Because it isn't a toy dataset problem — it's grounded in a real physical phenomenon with real downstream consequences (GPS positioning safety), required understanding both the domain science (ionospheric physics, geomagnetic storms) and the ML engineering (two full deep-learning architectures, storm-specific validation design, a proper modular codebase), and it produced a genuinely useful, literature-grounded conclusion (Transformer generalizes better to severe storms) rather than an inconclusive result.

Q50. What was your biggest contribution?

Answer: Designing and implementing the storm-specific validation methodology — deliberately holding out the three real storm days as the test set instead of using a random split — combined with building the full comparative LSTM-vs-Transformer modeling pipeline and the multi-metric (regression + event-detection) evaluation framework that made the comparison meaningful rather than superficial.

Q51. What mistake did you make during the project?

Answer: Early on, I let a CSV-overwrite bug silently drop previously processed data because I hadn't validated output row counts after each pipeline run — this taught me to always add lightweight sanity checks (row counts, value ranges) immediately after any data-transforming step, not just at the very end of the pipeline.

Q52. If you had to start over, what would you do differently?

Answer: I would design the modular src/ package structure from day one instead of starting in a single monolithic notebook and refactoring later, and I would include exogenous space-weather features (Kp, Dst, Ap) as model inputs from the start rather than treating them only as a post-hoc storm-classification tool.

Q53. How has this project prepared you for a software engineering role?

Answer: It gave me hands-on experience with the full lifecycle of a real engineering problem: turning a research prototype into a modular, reusable codebase; debugging real data-pipeline bugs (folder naming, label derivation, overwrite issues); designing evaluation methodology carefully rather than accepting default metrics; and clearly communicating technical trade-offs (architecture choice, metric choice) — all of which map directly onto day-to-day software engineering work, not just academic research.

Quick-Prep Checklist Before the Interview

Be ready to explain, fluently and without notes:

- Your project in 2–3 minutes, in plain language first, then technical depth if asked.
- The complete data → model → evaluation workflow, in order.
- Every technology/library you used, and specifically why you chose it over an alternative.
- Your individual contribution, phrased honestly relative to any teammates.
- At least two real challenges you faced and exactly how you solved them (use the CSV-overwrite bug and the LSTM smoothing issue as ready examples).
- At least two concrete future improvements, tied to a real limitation you've identified.